

社区先知 CommunityInsight · 备赛知识手册（完善版）

从零基础到把项目讲清楚（非程序员也能看懂 · 9·22 OPC 沙龙专用）

怎么用：**第 0 章**用一个例子 60 秒看懂全系统；**第一部分**补最小技术常识（全是生活比喻）；**第二部分**把项目从里到外讲透；**第三部分**教你怎么讲、怎么演；**第四部分**是“懂没懂”自测。★=必背，【评委爱问】=高频追问。对应冻结版本 **v1.0**，根目录 `C:\Users\wo'shuo'feng'su\Desktop\campus-insight-agent`。

第 0 章 · 60 秒看懂整个系统：跟着一条报修走一遍



不懂代码也没关系，跟着“王阿姨报修楼道灯坏了”走一遍，你就理解了前端、后端、智能体、数据库、转人工分别在干嘛。**把这条线能默讲出来，你就懂了半个项目。**

1. **王阿姨（居民端）** 打开小程序/网页，对着**接待员 Agent** 说：“我们 3 单元 2 楼灯不亮了，老人晚上怕摔。”
2. **接待员**做**意图识别**：判断这是“报修”，但信息不全（哪栋？严重吗？）→ **追问补全**：“请问是几号楼？是否影响通行？”——这就是感知与引导。
3. 信息齐了，接待员在**黑板**上发一条 `task_request` 给**报修调度员**。
4. **报修调度员**先查重（有没有人报过同一处，可**合并**），走**报修状态机**生成草稿，**生成正式工单前请王阿姨确认**（人不确认不建单）。
5. 王阿姨确认 → 工单**写进数据库**（手机号**加密**存、界面**打码**显示），状态=待派单，同时在黑板写 history、在 agent_logs **留痕**。
6. 系统发现“楼道照明+老人怕摔”涉及**安全隐患**，报修调度员**联动通知管理员**发一条安全提示，并**升级优先级**提醒网格员——这是多智能体**主动协作**。
7. **网格员端**的刘网格员收到待办，**网格助手**还给出“建议派给物业电工、附驳回话术”。
8. 全过程中，任何 Agent 想对用户说的话，都要先过 **Verifier（事实核对，比如工单号对不对）** 和 **Arbiter（合规>安全>专业>人工...）**，**合规审计员**有一票否决。
9. 如果哪一步机器判断不了（比如王阿姨情绪激动、诉求模糊），直接生成**人工处理包**（她是谁、经过哪些 Agent、卡在哪、要确认什么）转刘网格员，**不用她再重复一遍**。
10. 处理完 → 状态机走到“已反馈”，王阿姨收到结果并可评价，形成**闭环**。

一句话复盘：**界面收诉求 → 接待员分诊 → 专干 Agent 协作办事、写库留痕 → 校验仲裁把关 → 人在关键处确认/兜底 → 结果回到居民**。大模型只在“需要灵活表达”的少数环节出现，且失败自动回退规则，**断网这条主线照样能走**。

第一部分 · 必须知道的基础概念（人话版）

1.1 Web 应用三层结构（餐厅类比）

层	人话	餐厅类比	你的项目
前端	浏览器里看得到、点得到的界面	大堂、菜单、点餐屏	Vue3 + Naive UI, 三端
后端	收请求、做判断、读写数据、回结果	后厨、调度	FastAPI (Python, :8000)
数据库	长期存数据的仓库	仓库/账本	SQLite (一个文件, 可换 PostgreSQL)

一次操作：前端点"提交"→ 通过**接口**把数据给后端 → 后端判断、调度智能体、写库 → 把结果回前端 → 前端提示成功。

会讲这句即可："前端管交互，后端管逻辑和智能体调度，数据库管持久化，三者用接口通信。"

1.2 技术栈每个名词是什么、为什么选

- **FastAPI**: Python 主流后端框架，快、自带接口文档、和 AI 生态最契合。
- **Vue3**: 主流前端框架，把数据变成会动的页面；**Naive UI**: 现成组件库（按钮/弹窗/表单），保证三端统一；**Vite**: 前端启动与打包工具。
- **SQLite**: 一个文件即一个库，零安装、易备份、演示稳，读多写少的基层场景够用，**不是玩具**。
- **端口**: 一台电脑区分服务的门牌号——主服务 **8000**、扣子插件 api.py **18800**、Streamlit 备线 app.py **8501**。
- **双后端**: FastAPI 主力、Streamlit 早期备线，**共享同一服务层**，是演进留痕而非冗余。

1.3 接口/API/HTTP/JSON/Token

- **API/接口**: 后端的一个个"服务窗口"，有地址(URL)+动作(GET 查/POST 增改)。
- **HTTP 状态码**: 200 成、401 未登录、403 无权限、500 服务错。
- **JSON**: 前后端传数据的标准"键:值"格式，如 `{"success":true,"data":{...}}`。
- **Token 令牌登录**★: 登录成功发一张加密"门禁卡"，之后每次请求都带上，后端就知道你是谁、什么角色；有效期 12 小时，卡里还带 community 用于属地化。
- **三角色最小权限**: resident 居民 / grid 网格员 / elderly 老人，看到的端和能调的接口不同。

1.4 什么是智能体 Agent（和聊天机器人的本质区别）★

Agent = 自主完成 **感知**→**决策**→**行动**→**记忆** 闭环： - **感知**收诉求/事件；**决策**判意图、定下一步（规则为主、LLM 按需）；**行动**调真实工具（查库、建工单、发通知、联动角色）；**记忆**写黑板和数据库，可追溯可续接。

三句话区别（必背）：**聊天机器人**只输出文字、不改业务状态；**规则脚本**只会走写死分支、无协作无留痕；**你的 Agent** 有角色、能真办事、会协作、有校验仲裁和留痕、办不了转人——是“能办事、可审计”的智能体。

1.5 多智能体系统（MAS）

把复杂岗位拆成多个**专职角色**协作，像一个社区办公室。要解决三件事：**怎么共享信息（黑板）、怎么通信（消息协议）、意见不合听谁的（仲裁）**——你三样都有。为什么不用一个超级 Agent：①专职更可靠、可单测；②权责清晰可审计（政务刚需）；③单点故障不拖垮全局；④贴合真实基层组织、用户好懂。

1.6 LLM、规则引擎，为什么“规则为主、LLM 为辅”★

- **LLM 大模型**（DeepSeek）：理解/生成自然语言。**Prompt** 指令、**Token** 长度计费单位、**Temperature 温度**（0 最稳、越高越发散，你统一 0.1–0.3）、**幻觉**（一本正经编造，政务致命）。
- **规则引擎**：写死的确定逻辑，确定、零成本、毫秒级、可测，但不灵活。
- **你的路线**：规则为主干，LLM 只做增强，四个开关**默认全关**。
- 【评委爱问】“默认关是不是 AI 不够？”

“正因为是民生场景，我把**确定性和可控**放第一。状态流转、派单、权限、加密、升级阈值永远走规则；只有意图兜底、政策生成、协商演示这些‘要灵活表达’的环节才用 LLM，且每次都有超时、异常回落规则、强制引用和校验。**我能一键开增强，也能保证断网照常办事，这是主动选择的可信 AI 架构。**”

1.7 RAG（政策问答为什么不乱编）

- **RAG 检索增强生成**：先从**自己的知识库**检索真实材料 → 模型**只依据材料**组织回答 → **强制标注引用第几条**。
- 流程：提问 → **混合检索**（关键词 BM25 + 语义向量两路融合） → 取最相关几条 → LLM 只据此生成 → 校验引用下标真实 → 拼真实标题出处。
- **Embedding 向量化**：把文字变成语义坐标，意思近的坐标近，能按“意思”而非“字面”检索。
- **零结果红线**：检索不到材料**绝不生成、直接转人**。
- ★硬指标：42 条金标，混合检索 **hit@1=100%**，纯词法仅 85.7%。

1.8 知识图谱 KG

“实体—关系—实体”的网络（老人—患有—高血压—需注意—高温）。规模 **88 实体/199 关系/760 提及**。RAG 管“从材料找答案”，KG 管“实体间关联推理”，互补不重复。

1.9 意图识别/状态机/降级熔断/缓存

- **意图识别**：判断想干嘛。规则优先，全不命中才可选一次 LLM 兜底（白名单闭集，只能从固定意图选）。
- **状态机**：事情按固定阶段走、不能跳步。报修：提交→审核→派单→处理→反馈；提案：提交→审核→公示→投票→执行→反馈。
- **降级/熔断**：外部依赖（大模型/天气 API）超时或挂了，自动退回规则/缓存，主流程不崩。
- **缓存**：临时存复用结果（如天气），更快更省、外部抖动时顶上。

1.10 数据安全与合规（民生生命线）

- **加密落库**：手机号入库前用 **AES-256-GCM** 加密成密文，需用时后端解密。
- **脱敏展示**：界面/导出显示 138****6677。
- **HTTPS**：传输加密（方案已文档化，正式部署启用）。
- **PIPL 个人信息保护法**：最小必要、告知同意、安全保障、可查可删、留痕审计。
- **审计留痕**：谁/何时/对哪条数据/做了什么，全可查；Agent 决策写 agent_logs。
- **SQL 注入防护**：全部参数化，不拼接用户输入。

1.11 测试与压测（最硬的工程证据）★

- **pytest 单元测试**：给功能预写"考题"，改完一键全跑，改坏立刻红灯。
- ★ **581 项全绿**（另 1 跳过/3 反选/7 子测试/0 警告）；**550 并发零失败**；**ruff 0 问题**；**UI 54 视口 0 高危、移动端 21/21**。

"我每改一处都要让 581 个测试全绿才提交，系统是持续可回归验证的，不是堆出来的演示。"

1.12 适老化/无障碍

不是"字调大"：大字高对比大按钮留白防误触、**语音报修**、**长按紧急求助（防误碰）**、用药提醒、大字天气、危险操作二次确认的**防呆**、紧急症状直连就医/转人。方言现状要诚实（见 2.10）。

1.13 名词速查表

术语	人话	项目对应
前端/后端/数据库	脸/脑/记忆	Vue 三端 / FastAPI / SQLite
接口 API	服务窗口	约 130 个 HTTP 接口
Token	门禁卡	登录签发，带角色与社区
Agent	感知-决策-行动-记忆的办事员	9 角色
MAS	团队分工协作	Orchestrator 编排
Blackboard	共享白板+信箱	shared_data/messages/locks/history

术语	人话	项目对应
Arbiter	冲突时谁拍板	合规>安全>专业>人工>数据一致
Verifier	事实核查员	回查 DB、纠正幻觉
合规审计员	纪检、一票否决	compliance_auditor
LLM	会表达的外脑	DeepSeek, 默认关按需开
规则引擎	写死的靠谱流程	状态机/关键词/阈值
RAG	先查资料再答、标出处	policy_rag / rag.py
Embedding	语义坐标	混合检索语义路
知识图谱	实体关系网	88/199/760
降级熔断	外援掉了自己顶上	超时回落、天气缓存
AES	敏感数据进保险柜	手机号 phone_enc
脱敏	打码	138****6677
pytest	自动批改的 581 道题	tests/
状态机	流程按阶段走、不跳步	报修/提案生命周期

- 1.14 ★ 最容易被绕进去的概念辨析（想清楚就不慌）
- **多智能体 vs 固定工作流**：工作流是写死的"A 完了到 B"，路径不变；多智能体有**动态决策**（接待员按意图路由、仲裁按情况裁决、角色按事件主动联动），且角色间是**消息驱动**而非硬编码连线。
 - **多智能体 vs 一个大模型分饰多角**：后者只是 Prompt 里假装不同人，**没有独立状态、不能各自调工具、没有冲突仲裁和留痕**；你的角色是独立模块、各有工具与权限边界。
 - **RAG vs 微调 (fine-tune)**：RAG 是"开卷考试"，查现有资料作答、**资料更新即更新、可标出处**；微调是"把知识背进权重"，更新要重训、不可标出处。政策经常变，所以选 RAG。
 - **规则 vs LLM 不是对立**：规则保底确定与安全，LLM 提升表达与泛化，**叠加**才是民生正解。
 - **并发高 ≠ 性能好**：并发验证"很多人同时用不崩"，还要看单次响应快不快；你两者都有（550 并发 + 规则毫秒级）。
 - **加密 ≠ 脱敏**：加密是"存的时候锁起来、能解开"，脱敏是"显示时打码、不可逆"，两者都要。
 - **测试多 ≠ 质量高，但没有测试一定不可靠**：测试给你"敢改、能重构"的底气，这正是你能持续迭代十轮的原因。

第二部分 · 把项目彻底讲清楚

- 2.1 ★ 一句话定位与痛点
- 社区先知是面向社区/街道的"接诉即办"网格化多智能体平台：9 个分工协作的 AI 角色服务居民、网格员、老年人三类人，把报修、提案、通知、政策问答、健康咨询、天气预警、紧急求助做到"AI 先办、

合规把关、人工兜底、全程留痕", 并专门适老化。

痛点：①基层事多人少、网格员被重复事务淹没；②传统系统"人找系统"、被动；③老人被数字服务落下；④通用大模型直接上政务有幻觉/不可控/不合规/断网就废四风险。价值 = 主动服务 + 智能分流 + 可信可控 + 适老兜底。

2.2 总体架构（要能在白板画出来）



2.3 技术选型与理由（为什么这么选）

选择	理由
FastAPI 起步、按模块拆路由	AI 生态好、开发快；已拆出十余个路由模块，不过度设计、按需演进
SQLite 先行	零运维、演示稳、单街道够用；数据层抽象，可平滑换 PostgreSQL
Vue3+Naive UI	组件统一、便于设计令牌/暗色/对比度/适老
规则为主+LLM 为辅	政务要确定、可审计、低成本、断网可用
自研协作内核	完全掌控消息/仲裁/留痕/离线，不被外部框架绑死（规模上来可换 Redis 队列，接口已留）
DeepSeek	国产合规友好、成本低、够用

2.4 ★ 九个智能体逐个讲

key	中文名	职责	人工兜底点
receptionist	社区接待员	统一入口：意图识别、追问、情绪安抚、纠错、路由	意图不明转人
repair_dispatch	报修调度员	报修全流程（草稿/创建/派单/跟踪/反馈）	建工单前必须用户确认
proposal_collab	提案协商员	提交/公示/投票/执行/反馈	执行须负责人批准
health_advisor	健康顾问	健康咨询、就医指引、预防提醒（不诊断）	紧急症状提示就医/转人
policy_expert	政策专员	政策问答，强制引用、无引用不答	无引用转人
notification_manager	通知管理员	通知创建/定时/发布	紧急通知二次确认，Agent 不直发
weather_guardian	天气守护员	监测天气、触发预警、联动健康与通知	不自动执行应急措施
grid_assistant	网格员助手	搜索/导出/统计/待办/跳转/驳回建议	只辅助不拍板
compliance_auditor	合规审计员	脱敏/敏感词/权限，最终否决权	合规问题直接拦截

记忆：1 前台 + 5 专干 + 1 自动值守 + 1 网格员副驾 + 1 纪检。

2.5 ★★ 核心亮点：多智能体怎么协作（与套壳的分水岭）

黑板 Blackboard 四件套：shared_data（共享键值，带锁带版本带时间戳）、messages（消息队列）、locks（写锁防并发冲突）、history（全量历史供执行链/审计/复盘）。

五种消息协议：task_request / task_response / notify / error / handoff——结构化、可记录、可重放，不是互相“喊话”。

三个真实协作场景（Demo 演这几个）：1. **天气↔健康（主动）：**天气守护员发现极端天气 → 请健康顾问出老人防护建议 → 通知管理员发提醒，全程留痕，**用户没问系统主动服务**。2. **报修→通知（隐患升级）：**识别为电梯/消防等隐患 → 联动发安全提示、升级优先级、提醒负责人。3. **llm_collaboration（可选）：**开关打开时编排决策器输出结构化 JSON（联动谁、为什么），过校验才生效。

Orchestrator 编排器：接事件、分发、驱动消息流、限制最大迭代 6 次（防角色互相踢皮球死循环）。

金句：“不是九个独立聊天框，而是有共享状态、标准通信、主动联动、并发安全、全程留痕的协作系统。”

2.6 ★ 双层防线：为什么是“可信 AI”

- **Verifier 事实校验：**回复里的工单号/提案号回查数据库核对、纠正编造编号；只补充不擅改正文。

- **Arbiter 仲裁固定优先级**：合规→安全→专业→人工→数据一致→默认保守；裁决 block/human/professional/latest_version/default；跨部门优先级里合规=1、一票否决。
- **合规审计员**：敏感词、脱敏、权限，输出前贯穿。

"AI 可以提建议，但能不能说出口要先过事实校验和合规仲裁；拿不准、涉安全合规一律转人。"

2.7 LLM 用在哪、怎么防风险（边界要精确）

- **就四处用 LLM**：意图二级兜底、政策 RAG 生成、健康建议润色、可选自主协商。
- **四开关默认全关**：LLM_ORCHESTRATION / LLM_NEGOTIATION / POLICY_LLM_RAG / RECEPTION_LLM_FALLBACK。
- **防幻觉五层**：强制 RAG+引用（下标必须真实否则废弃转人）、意图白名单闭集、协商只出结构化 JSON 且目标在候选集、低温限长、输出再过 Verifier/合规（不过回退规则）。
- **防 Prompt 注入五层**：输入隔离、指令与数据分离、长度限制、敏感模式拦截、输出侧校验。
- **可靠性**：统一 llm_client，每次 3-10s 超时+熔断+异常回落规则+用量记录；没 Key/断网/限流都不白屏。

2.8 十大业务模块（各一句话）

①报修完整状态机+重复合并+草稿落库(7天清理)；②提案全流程含重新执行；③通知普通/紧急/定时+已读统计+紧急弹窗+二次确认；④政策知识库版本控制+自动答+人工兜底+强制引用+属地加权；⑤健康内容/咨询/就医指引/紧急转人；⑥天气预警→预防提醒→核查任务+缓存降级；⑦老年端语音/大字天气/用药提醒/长按求助/防呆；⑧网格员工作台搜索导出统计+待办+驳回建议；⑨转人工"处理包"（用户信息+执行链+待确认）；⑩Agent 统一入口三端对话+快捷问题+历史+限流。

2.9 数据层：为什么改库走"迁移"不直接 ALTER

- 结构用**版本化迁移**管理，当前 **v46**（db_core.py 用 _m{N}_ 注册每次变更）。
- 【评委爱问】"为何不直接改表？"："生产库有真实数据，直接改易丢数据、不可追溯、别人机器对不上版本；迁移保证任何旧库都能可重复、可审计地升到最新。"
- 约 50 表、开 **WAL**（读写并发更稳）。
- **手机号铁律**：加密落库(phone_enc, AES-256-GCM)、用时后端解密、展示导出脱敏(_enc/_dec/_mask)；已核验 8 张相关表**明文残留 0**。

2.10 三端与适老化（细节越具体越可信）

- 居民端：首页/报修/提案/通知/我的，情绪安抚、草稿保护。
- 网格员端：深色侧导航+工作台+AI 助手抽屉（默认收起）。
- 老年端：超大字、高对比、语音、长按求助、用药提醒、极简路径。
- 全局：设计令牌、暗色、AA 对比度、54 视口自适应、移动 21/21。
- 诚实边界：语音以普通话为主，**方言词表/指代消解/否定模糊是规划项**，不吹已完成。

2.11 转人工闭环（人在回路）

触发转人自动打包：脱敏用户信息 + **完整执行链**（经过哪些 Agent、判断与动作、黑板消息）+ **待确认事项**；人工一键接手不让居民重复说，处理完回填闭环。

2.12 地区识别（属地化，已落地）

- 不做 GPS/IP（过度设计、老人授权负担、隐私面），**按所属 community 解析属地**。
- 政策**属地加权**只调排序不改自动答阈值（base/final 双分，防属地误顶）；天气按 community 的 city_id 透传。
- 【评委爱问】"怎么知道用户在哪？"："用治理网格里本就有的所属社区，不额外采集位置，更准也更护隐私。"

2.13 ★ 工程质量证据清单

维度	证据
功能正确	581 pytest 全绿、0 警告，改完必全绿
高可用	550 并发零失败、超时熔断缓存降级
规范	ruff 0
前端	54 视口 0 高危、移动 21/21、暗色、AA
检索	RAG 42 金标 hit@1 100% vs 词法 85.7%
知识结构	KG 88/199/760
安全	AES-256-GCM、8 表手机号 0 明文、参数化 SQL、最小权限
可追溯	agent_logs、history、执行链、仲裁留痕
可演进	v46 迁移、数据层抽象可换 PG、模块化拆路由

2.14 ★★ 必背数字

9 角色、3 端；**581** 测试全绿、**550** 并发零失败；库迁移 **v46**、约 **50** 表、约 **130** 接口；RAG **42** 金标 hit@1 100%（词法 85.7%）；KG **88/199/760**；**AES-256-GCM**、**8** 表手机号 0 明文；仲裁 6 级；消息 5 种；4 个 LLM 开关默认关、超时 3–10s 失败回落。演示账号：居民 **demo_resident** 免密(王阿姨)、网格 **demo_grid/demo123** (刘网格员)、老人 **demo_elderly** 免密(张大爷)。

2.15 ★ 设计取舍："我为什么不做 X"

- **不全开 LLM**：确定性核心不能赌模型，全开=慢/贵/不可复现/断网崩。
- **不做 GPS**：所属社区已足够，不增授权与隐私负担。
- **用 SQLite**：单街道够、零运维、演示稳，可平滑换 PG 不返工。
- **自研不套现成框架**：要完全掌控消息/仲裁/留痕/离线，队列可后换 Redis。

- **保留 Streamlit 备线**：内部快速核查兜底，共享服务层成本极低。
- **Agent 不直发紧急通知/不自动应急**：高风险动作必须人确认，安全红线。

2.16 项目是怎么一步步迭代出来的（讲"成长故事"很加分）

评委会问"你这是一次堆出来的还是迭代出来的"。你要能讲出**演进脉络**（具体可看 [docs/spec/dev-log.md](#)）：1. 先搭三端+报修/提案/通知主流程跑通闭环；2. 再加 9 角色声明化、黑板、消息协议、仲裁、校验，形成多智能体内核；3. 补安全：手机号加密迁移、脱敏、安全头、参数化、Prompt 注入防护；4. 补智能：RAG 混合检索+金标评测、知识图谱、LLM 按需增强（默认关）；5. 补工程：测试从几百项涨到 581、并发压测、ruff、UI/移动审计、模块化拆路由；6. 补适老与人情味、属地化、转人工上下文，最终**冻结 v1.0**。金句："我不是一上来求大而全，而是每一轮都先补最关键的短板、用测试守住不退化，前后打磨了十轮才冻结。"

2.17 差异化：和三类"对手"比

对比对象	它的特点	你的差异/优势
通用大模型直接对话（ChatGPT 类）	会说但不办事、会幻觉、无业务状态、无权限、断网废	你能真调工具办事、状态机闭环、强制引用、合规仲裁、转人、离线可跑、留痕
固定 workflows/传统数字政务	被动受理、人工流转、偏台账	AI 前置分流+主动联动+多角色协同+适老，是"智能增效层"，可对接不替代
大厂智慧社区方案	重、贵、部署周期长、偏硬件平台	你轻量、低成本、快速试点、软件智能协同与适老更细，可做其前端智能补充，错位竞争
学生常见"Agent 项目"	单一大模型+几个 Prompt、无测试、依赖外部 Key 现场易崩	真协作内核+581 测试+550 并发+加密+断网可用，完成度与稳定性碾压

2.18 已知短板与诚实说法（主动说反而加分）

短板	专业说法
无真实街道试点	完整可运行原型+演示数据，下一步对接 1 社区小范围试点
多租户未全表化	已有 community 字段级隔离，tenant_id 是生产化排期
12345/舆情外部源占位	已留对接适配层，需真实授权接入
系统级 traceId	业务执行链已有，request_id 全链路是下一步可观测性
方言/指代/否定 NLU	规则+白名单覆盖高频，方言词表与指代消解在路线图
LLM 评测集	已有 42 条 RAG 金标，将扩意图/协商 golden set
商业模式较薄	政府购买+SaaS，先标杆后复制（见问答库）
> 原则： 短板不藏，但每个都跟一句"我准备怎么解决" 。	

第三部分·怎么讲、怎么演

3.1 ★ 三版陈述

- **30 秒**："我做的是社区接诉即办的多智能体平台，九个分工 AI 角色通过共享黑板协作，服务居民、网格员、老人。能真建工单发通知、主动预警，办不了带完整上下文转人，全程留痕、数据加密。最大特点是规则为主大模型为辅、可信可控、断网能跑，581 项测试全绿。"
- **1 分钟**：加天气↔健康联动例子 + 双层防线 + 强制引用 + 适老 + 550 并发/v46/可换 PG。
- **3 分钟**：痛点(30s)→方案(40s)→亮点(60s)→证据(30s)→规划(20s)。

3.2 ★ Demo 动线（自己点 10 遍形成肌肉记忆）

1 登录页看指标 → 2 网格端 `demo_grid/demo123` 工作台+AI 助手 → 3 居民端 `demo_resident` 报修（草稿/状态）+政策（带引用）→ 4 **王牌：天气→健康→通知的黑板消息流与执行链** → 5 老年端 `demo_elderly` 大字/语音/长按求助 → 6 转人工看"处理包"。全程口播"这步规则在办/这步 AI 增强/这步被合规拦下转人"，把**可控**演出来。

3.3 断网/翻车兜底

录 2-3 分钟本地操作视频+关键页截图+测试全绿终端照片；提前开三端三标签登录好；真断网就演规则流程并强调"断网照样办事"。

3.4 表达框架（万能结构）

- **结论先行**："结论是 X，我分两点说。"
- **STAR 讲功能**：情境(S)→任务(T)→怎么做(A)→结果/数据(R)。
- **被质疑**：先认可（"这个问题很关键"）→ 给事实/设计 → 给规划。
- 不会就坦诚+转请教，**绝不编技术名词**。

第四部分·懂没懂？自测 30 问（能脱口答才算掌握）

盖住答案自问自答，卡住的回对应章节。全部能答，一对一就不慌。

1. 一句话说清项目是干嘛的？(2.1)
2. 三类用户各自最痛的点？(2.1)
3. 前端/后端/数据库分别什么作用？(1.1)
4. 什么是 Agent，和聊天机器人区别？(1.4)
5. 为什么要多个 Agent 而不是一个？(1.5)
6. 九个角色各管什么、谁是否决权？(2.4)
7. 黑板四件套？为什么需要锁？(2.5)
8. 五种消息协议？(2.5)
9. 讲一遍天气怎么联动健康？(2.5)
10. 仲裁优先级顺序、几种裁决结果？(2.6)
11. Verifier 和 Arbiter 分工？(2.6)
12. LLM 用在哪四个环节、开关为何默认关？(2.7/1.6)

5. 什么是 RAG，怎么防幻觉，金标多少？ (1.7) 14. 检索不到材料怎么办？ (1.7) 15. 知识图谱和 RAG 区别？ (1.8/1.14)
6. 报修/提案状态机各阶段？ (1.9/2.8) 17. 大模型挂了系统会怎样？ (2.7) 18. 手机号怎么保护、加密脱敏区别？ (1.10/2.9)
7. 为什么改库走迁移、现在第几版？ (2.9) 20. 为什么用 SQLite、以后怎么换 PG？ (2.15) 21. 转人工处理包里有什么？ (2.11)
8. 地区怎么识别、为什么不用 GPS？ (2.12) 23. 581 测试/550 并发说明什么？ (1.11) 24. 老年端除了大字还有什么？ (1.12/2.10)
9. 你和通用大模型/传统政务/大厂方案差异？ (2.17) 26. 项目最大短板、怎么补？ (2.18) 27. 下一步三件事？ (问答 B-Q24)
10. 代码是 AI 写的你怎么答？ (问答 B-Q21) 29. 演示主线顺序？ (3.2) 30. 必背数字一口气说全？ (2.14)
-

附：看源码的顺序（想再懂一点）

AGENTS.md（说明书）→ README.md（概览）→ agent/orchestrator.py（编排）→ blackboard.py/arbiter.py/verifier.py（协作与防线）→ agent/roles/config.py（9 角色）→ data/db_core.py（schema/v46）→ docs/spec/dev-log.md（迭代史）。

不用看懂每行代码，但要能对每个文件说出“它负责什么、为什么需要它”。